SecureGate AI

DevSecOps Security Decision Platform

Official Project Documentation

SecureGate AI Documentation

Continuous Security Validation For CI/CD Pipelines

CI/CD Security Gate | Reports | Alerts | Grafana | AI Remediation

Project:	**SecureGate AI**
Version:	**Sprint 6 Completion**
Prepared For:	**Capstone Project Evaluation**
Prepared By:	**SecureGate AI Project Team**
Date:	**July 31, 2026**

All Rights Reserved

Project Documentation Index

This PDF is generated from the official SecureGate AI Markdown documentation. The index below maps to the major documentation sections.

No.	Section
1	Introduction
2	Scope
3	Audience
4	Acknowledgement
5	Business Aspect
6	Business Problem
7	Business Value
8	Architectural Overview
9	Operating Model
10	Tools And Technologies
11	Security Gate Policy
12	Data Model And Storage
13	Reports And Documentation Outputs
14	AI Assistant And Remediation Engine
15	Alerts, Notifications, And Summary Jobs
16	Monitoring And Visualization
17	Deployment
18	CI/CD Manifests
19	Configuration Reference
20	Validation
21	Operational Runbook
22	References
23	Summary
24	Author

Copyright © 2026 SecureGate AI

Introduction

SecureGate AI is an enterprise DevSecOps security decision platform designed to help teams validate application security before deployment. It receives scan results from CI/CD pipelines, normalizes findings from multiple tools, calculates a security score, determines whether deployment should be approved or rejected, generates reports, sends notifications, visualizes trends, and provides AI-assisted remediation guidance.

The platform is built around a clear separation of responsibilities. CI/CD systems such as GitHub Actions and Jenkins execute the scanners. SecureGate AI receives the reports through APIs and acts as the decision, reporting, monitoring, and remediation layer.

SecureGate AI answers one operational question:

Is this repository safe enough to build, package, push, and deploy?

The final implementation is complete through Sprint 6 and includes dashboard views, project registry, scan history, report exports, alerts, Grafana monitoring, and local AI remediation support.

Scope

The project scope includes the following capabilities:

- Register one SecureGate AI project per GitHub repository.
- Enforce strict repository uniqueness so the same repository URL cannot be registered as multiple projects.
- Accept scan metadata from CI/CD pipelines.
- Receive uploaded reports from GitLeaks, Semgrep, Trivy filesystem, Trivy image, SonarQube or SonarCloud, and unit-test stages.
- Store raw report files and normalized findings.
- Calculate security score and deployment decision.
- Generate master reports in PDF, DOCX, and JSON formats.
- Generate readable PDF versions of individual tool reports.
- Generate AI security summaries and AI remediation plans.
- Display dashboard, project, scan, reports, remediation, monitoring, alerts, analytics, settings, user guide, and manifest pages.
- Send Slack and email notifications for pass/fail events and severity-aware alerts.
- Send daily, weekly, and monthly summary notifications through scheduled GitHub Actions workflows.
- Expose Prometheus metrics and provision Grafana dashboards.
- Support local AI analysis through Ollama and Gemma with fallback logic when the model is unavailable.

The following items are outside the current production scope:

- Direct scanner execution by SecureGate AI.
- Replacing GitHub Actions or Jenkins as the CI/CD runner.

Copyright © 2026 SecureGate AI

- Cloud-hosted LLM dependency.
- Automatic production deployment approval outside configured CI/CD policy.
- Full enterprise identity management such as SSO or RBAC.
- Multi-tenant SaaS isolation.

Audience

This documentation is intended for:

- DevOps engineers integrating SecureGate AI into CI/CD pipelines.
- Security engineers reviewing vulnerability, secret, and quality gate results.
- Developers using remediation plans to fix scan findings.
- Engineering managers reviewing project health and deployment readiness.
- Instructors and evaluators reviewing the capstone implementation.
- Platform administrators responsible for Docker Compose, environment variables, Grafana, Prometheus, Slack, email, and SonarCloud integration.

Acknowledgement

This project uses widely adopted open-source and industry security tools including FastAPI, React, PostgreSQL, Docker, GitLeaks, Semgrep, Trivy, SonarQube/SonarCloud, Prometheus, Grafana, Alertmanager, Ollama, and Gemma. Their documentation, community practices, and tool outputs guided the platform design.

SecureGate AI was developed as a capstone project to demonstrate end-to-end DevSecOps automation, reporting, monitoring, and AI-assisted remediation.

Business Aspect

Modern software teams ship changes frequently through automated CI/CD pipelines. Security validation must keep pace with this delivery model. Manual security reviews are slow, inconsistent, and difficult to audit. Individual scanner dashboards provide useful data, but they often do not give one clear deployment decision across all tools.

SecureGate AI centralizes this security decision process. It allows teams to keep using their existing CI/CD runners while adding one standardized security governance layer.

Key business goals:

- Reduce risk before code reaches production.
- Standardize security gates across repositories and branches.
- Improve visibility for managers, DevOps teams, and developers.
- Reduce manual reporting effort.
- Preserve audit history for security and compliance review.

Copyright © 2026 SecureGate AI

- Provide actionable remediation guidance instead of only raw scanner output.

Business Problem

Containerized and CI/CD-driven applications often rely on several separate tools:

- Secret scanning tools identify exposed credentials.
- Static analysis tools identify insecure code patterns.
- Dependency scanners identify vulnerable packages.
- Image scanners identify operating system and package vulnerabilities.
- Code quality tools identify maintainability and quality gate risks.
- CI/CD logs identify build, test, and packaging failures.

The problem is that these results are usually scattered across different systems. DevOps teams may need to check GitHub Actions logs, SonarCloud dashboards, Trivy JSON files, Docker build output, Slack messages, and manually generated reports. This creates several risks:

- Security findings may be missed.
- Deployment decisions may be inconsistent.
- Developers may not know what to fix first.
- Managers may not have a clear project health view.
- Historical trend analysis becomes difficult.
- Audit evidence is hard to collect.

Business Value

SecureGate AI provides measurable value by transforming separate scan outputs into a single decision and remediation workflow.

Business value includes:

- Faster release decisions through automated scoring and approval/rejection logic.
- Stronger security posture through strict gates on all branches.
- Reduced production risk by blocking secrets, critical vulnerabilities, quality gate failures, and policy violations.
- Better developer productivity through prioritized remediation plans.
- Better management visibility through dashboards, reports, and summary notifications.
- Better compliance posture through stored audit logs, reports, and scan history.
- Lower operational complexity by supporting existing GitHub Actions and Jenkins pipelines.
- Improved incident response through severity-aware alerts and escalation visibility.

Copyright © 2026 SecureGate AI

Architectural Overview

SecureGate AI follows a report-ingestion and decision-engine architecture.

```
Developer pushes code to any branch
        |
GitHub Actions or Jenkins pipeline starts
        |
Git checkout
        |
GitLeaks secret scan
        |
Semgrep static analysis
        |
Trivy filesystem scan
        |
SonarQube or SonarCloud analysis
        |
Unit tests when available
        |
Docker build when policy allows
        |
Trivy image scan
        |
Optional Docker Hub push when approved and credentials exist
        |
Reports uploaded to SecureGate AI APIs
        |
FastAPI normalizes and stores findings
        |
Security score and deployment decision are calculated
        |
Reports, alerts, notifications, dashboard, Grafana metrics, and AI guidance are generated
```

Core Components

Component	Responsibility
React Frontend	Provides dashboards, project pages, scan details, reports, alerts, analytics, monitoring, manifests, settings, and AI views.
FastAPI Backend	Receives reports, normalizes findings, calculates decisions, generates reports, exposes APIs, sends notifications, and serves metrics.
PostgreSQL	Stores projects, scans, tool reports, findings, reports, audit logs, notifications, alerts, and custom alert policies.
Storage Directory	Stores raw uploaded JSON reports and generated report files.
GitHub Actions/Jenkins	Executes scanners and uploads results to SecureGate AI.
Prometheus	Scrapes SecureGate AI metrics.
Grafana	Visualizes scan, finding, alert, report, and notification trends.
Alertmanager	Supports monitoring alert flow.
Ollama/Gemma	Provides local AI-assisted summaries and remediation guidance.

Operating Model

SecureGate AI uses the following operating model:

1. A user registers a project in the dashboard.
2. SecureGate AI generates a project ID such as `PRJ-YYYYMMDD-0001`.

Copyright © 2026 SecureGate AI

3. The user copies a CI/CD manifest from the Manifests page.
4. The user adds the manifest to the source repository.
5. The CI/CD pipeline runs automatically on pushes to any branch or manually through workflow dispatch.
6. The pipeline executes security tools and uploads reports.
7. SecureGate AI creates a scan ID such as `SCAN-YYYYMMDD-0001`.
8. The backend stores reports and normalized findings.
9. The security gate calculates score and decision.
10. Reports and dashboards are updated.
11. Slack and email notifications are sent.
12. AI summary and remediation plan are generated for the scan.
13. Grafana and Prometheus reflect operational and security metrics.

Tools And Technologies

Frontend

Tool	Purpose
React	Application UI framework.
Vite	Frontend build and development server.
TypeScript	Typed frontend development.
Lucide Icons	Consistent dashboard iconography.
CSS	Custom enterprise dashboard styling.

Backend

Tool	Purpose
FastAPI	REST API backend.
Python	Backend implementation language.
SQLAlchemy	Database models and ORM.
Pydantic	Request and response schemas.
ReportLab / DOCX tooling	PDF and DOCX generation.

Database And Storage

Tool	Purpose
PostgreSQL	Primary relational database.
Docker named volumes	Persistent service data.

Copyright © 2026 SecureGate AI

Tool	Purpose
Local storage directory	Raw uploaded reports and generated report files.

Security Tools

Tool	Purpose
GitLeaks	Detects secrets and credentials in source code.
Semgrep	Detects insecure code patterns and static analysis findings.
Trivy FS	Scans source filesystem dependencies and manifests.
Trivy Image	Scans built Docker images for vulnerabilities.
SonarQube/SonarCloud	Provides code quality, maintainability, security, coverage, and quality gate results.
Unit Test Stage	Captures whether tests exist and pass when available.

Monitoring And Notifications

Tool	Purpose
Prometheus	Metrics collection.
Grafana	Visualization dashboards.
Alertmanager	Alert routing support.
Slack Webhook	Real-time team notifications.
SMTP Email	Email alerts and summaries.

AI

Tool	Purpose
Ollama	Local LLM runtime.
Gemma 3 4B	Local model used for summaries and remediation guidance.
Rule-based fallback	Deterministic response when Ollama is unavailable.

Security Gate Policy

SecureGate AI applies a strict security policy. Deployment is rejected when any blocking condition is true.

Default Blocking Rules

Copyright © 2026 SecureGate AI

Rule	Result
Any GitLeaks secret exists	Rejected
Any critical vulnerability exists	Rejected
High vulnerability count exceeds configured threshold	Rejected
SonarQube/SonarCloud quality gate fails when required	Rejected
Security score is below threshold	Rejected
Required pipeline stage fails	Rejected
Docker image build or image scan is missing when required by policy	Rejected or incomplete readiness

Default Score Model

Finding Type	Impact
Critical vulnerability	Major score deduction and production blocker.
High vulnerability	Strong score deduction and may block by threshold.
Medium vulnerability	Moderate deduction and remediation required.
Low vulnerability	Light deduction and monitoring required.
Secret finding	Automatic blocker.
Failed quality gate	Automatic blocker.

Default passing score: `80`.

P1-P7 Remediation Priority Logic

Priority	Meaning	Typical Action
P1	Emergency production blocker	Remove exposed secrets or active credential leaks immediately.
P2	Critical security blocker	Fix critical vulnerabilities before release.
P3	Quality gate or policy blocker	Resolve SonarQube/SonarCloud gate failure or release policy violation.
P4	Critical CVE remediation	Patch critical filesystem or image CVEs.
P5	High vulnerability remediation	Fix high-risk vulnerabilities quickly.
P6	Important security hygiene	Resolve code smells, maintainability risks, repeated warnings, or non-blocking tool issues.
P7	Lower urgency but tracked remediation	Fix medium/low technical debt in the next remediation cycle.

Copyright © 2026 SecureGate AI

P7 does not mean the issue can be ignored. It means the issue is lower urgency than P1-P6 but remains tracked until resolved.

Data Model And Storage

SecureGate AI uses PostgreSQL as the primary database.

Important tables include:

Table	Purpose
projects	Registered project metadata and repository identity.
scans	Individual scan executions and decisions.
tool_reports	Uploaded raw report metadata.
findings	Normalized findings from all tools.
reports	Generated master, tool, AI summary, and remediation report records.
audit_logs	Timeline of scan, upload, scoring, report, alert, and AI events.
notifications	Slack and email delivery history.
alerts	Severity-aware alert records and escalation state.
project_alert_policies	Project-specific custom alert overrides.

Raw reports and generated files are stored under the backend storage directory:

```
storage/scans/{scan_id}/
```

Example files:

```
gitleaks-report.json
semgrep-report.json
trivy-fs-report.json
trivy-image-report.json
sonarqube-report.json
unit-tests-report.json
deployment_approval.pdf
deployment_approval.docx
ai-summary.pdf
ai-remediation-plan.pdf
```

Reports And Documentation Outputs

SecureGate AI generates several report types.

Report	Format	Purpose
Master Security Report	PDF, DOCX, JSON	Manager and DevOps overview of score, decision, findings, tool status, recommendations, and summary.
Tool Reports	JSON, PDF	Raw and readable views of each uploaded scanner report.
AI Security Summary	PDF	AI-assisted executive scan explanation.

Copyright © 2026 SecureGate AI

Report	Format	Purpose
AI Remediation Plan	JSON, PDF	Developer-focused prioritized fix plan.
Summary Notifications	Slack, Email	Daily, weekly, and monthly scan activity summaries.

Reports include project name, repository link, branch, upload time, scan ID, score, decision, tool status, severity counts, top findings, summary, and recommendations.

AI Assistant And Remediation Engine

Sprint 6 adds local AI-assisted remediation. The AI system uses stored scan context and normalized findings to generate practical guidance.

AI capabilities include:

- Ask AI Assistant floating widget.
- AI Summary on dashboard and project pages.
- AI Security Summary PDF.
- AI Remediation Plan PDF.
- Project-level remediation page.
- P1-P7 priority matrix.
- Estimated fix time.
- Production readiness checklist.
- Rule-based fallback when Ollama is unavailable.

The AI assistant can answer questions such as:

- Why did the latest scan fail?
- What should I fix first?
- Which files are affected?
- Is the project ready for deployment?
- What is the estimated remediation effort?

The AI engine is designed to support developers without replacing security policy. The final deployment decision still comes from deterministic SecureGate AI security rules.

Alerts, Notifications, And Summary Jobs

SecureGate AI supports real-time and scheduled communication.

Real-Time Notifications

Notifications can be sent for:

Copyright © 2026 SecureGate AI

- Scan approved.
- Scan rejected.
- Critical severity detected.
- High severity threshold reached.
- Medium risk requiring remediation.
- Quality gate failure.
- Secrets detected.
- Alert escalation events.

Channels:

- Slack webhook.
- SMTP email.
- Dashboard notification history.

Default Alert Routing Policy

When no custom project alert policy exists, SecureGate AI uses default routing:

Severity	Dashboard	Slack	Email
Critical	Yes	Yes	Yes
High	Yes	Yes	Yes
Medium	Yes	Yes	Optional by configuration
Low	Yes	Optional	Optional

Custom Alert Management

The Alerts page includes project-specific custom alert management. A user can choose a project and configure:

- Whether custom policy is enabled.
- Slack notification enabled or disabled.
- Slack minimum severity.
- Email notification enabled or disabled.
- Email minimum severity.

When enabled, the custom project policy overrides the default Alert Routing Policy for future scans and future alerts. When disabled, default routing applies.

Scheduled Summaries

The summary workflow supports:

- Daily summary.

Copyright © 2026 SecureGate AI

- Weekly summary.
- Monthly summary.

These are triggered by GitHub Actions cron using `examples/github-actions/securegate-summary.yml`.

Summary messages include:

- Projects scanned.
- Passed scans.
- Failed scans.
- Critical, high, medium, and low counts.
- Pending and resolved counts.
- Project-level vulnerability breakdown.
- Simple action summary.

Monitoring And Visualization

SecureGate AI exposes Prometheus metrics at:

```
http://localhost:8000/metrics
```

Prometheus scrapes these metrics and Grafana visualizes them.

Grafana URL:

```
http://localhost:3000
```

Default credentials:

```
admin / admin
```

Current Grafana dashboard:

```
SecureGate AI - Executive Security Dashboard
```

Current monitoring coverage includes:

- Scan counts.
- Completed and failed scans.
- Findings by severity.
- Reports generated.
- Notification health.
- Alerts and escalation state.
- Service readiness.
- CI/CD health.

Future Grafana improvements can include:

Copyright © 2026 SecureGate AI

- Project Security Trends.
- Tool-Level Findings.
- CI/CD Health.
- Notification Delivery Health.
- Docker Image Risk.
- Branch-level drilldowns.
- Image-level vulnerability drilldowns.

Deployment

Prerequisites

Install:

- Docker Desktop or Docker Engine with Docker Compose.
- Git.
- GitHub repository access.
- Optional: ngrok for local public webhook testing.
- Optional: SonarCloud account.
- Optional: Docker Hub account.
- Optional: Slack webhook and SMTP email account.

Project Directory

Recommended local path:

```
cd /Users/saimausman/Desktop/securegate-ai
```

Start All Services

```
docker compose up -d --build
```

Check Service Status

```
docker compose ps
```

Backend Health Check

```
curl -fsS http://localhost:8000/health
```

Expected response:

```
{"status":"healthy"}
```

Pull AI Model

```
docker compose exec ollama ollama pull gemma3:4b
```

Copyright © 2026 SecureGate AI

Check AI Model

```
docker compose exec ollama ollama list
```

Rebuild Frontend And Backend After Code Changes

```
docker compose up -d --build frontend backend
```

Restart Backend Only

```
docker compose restart backend
```

View Logs

```
docker compose logs -f backend

docker compose logs -f frontend

docker compose logs -f ollama
```

Fresh Database Reset

Use this only when starting from a clean demo environment.

```
docker compose down -v
docker compose up -d --build
docker compose exec ollama ollama pull gemma3:4b
```

This removes PostgreSQL, Prometheus, Grafana, SonarQube, and Ollama volumes.

CI/CD Manifests

SecureGate AI provides reusable CI/CD manifests.

Runner	Manifest Path
GitHub Actions Scan Pipeline	`examples/github-actions/securegate.yml`
GitHub Actions Scheduled Summary	`examples/github-actions/securegate-summary.yml`
Jenkins Pipeline	`examples/jenkins/Jenkinsfile.securegate`
Frontend Manifest Library	`http://localhost:5173/manifests`

GitHub Actions Workflow Behavior

The scan workflow runs on pushes to any configured branch and through manual workflow dispatch. It performs:

1. Git checkout.
2. SecureGate AI pipeline start.

Copyright © 2026 SecureGate AI

3. GitLeaks scan.
4. Semgrep scan.
5. Trivy filesystem scan.
6. SonarQube/SonarCloud analysis.
7. Unit test stage when applicable.
8. Docker build when policy allows or fallback Dockerfile generation is available.
9. Trivy image scan.
10. Optional Docker Hub push when credentials exist and policy allows.
11. Tool report uploads.
12. Master report generation.
13. Notification sending.
14. Pipeline completion update.

Scheduled Summary Workflow

The summary workflow supports daily, weekly, and monthly summary messages.

Typical secrets required:

```
SECUREGATE_URL
SECUREGATE_API_KEY
```

Required GitHub Secrets For Full Scan Pipeline

```
SECUREGATE_URL
SECUREGATE_API_KEY
SONAR_TOKEN
SONAR_HOST_URL
SONAR_PROJECT_KEY
SONAR_ORGANIZATION
DOCKERHUB_USERNAME
DOCKERHUB_TOKEN
```

Docker Hub secrets are optional. Without Docker Hub credentials, the pipeline can still scan and report, but image push is skipped.

Configuration Reference

Create the environment file at:

```
/Users/saimausman/Desktop/securegate-ai/.env
```

Recommended values:

```
SECUREGATE_API_KEY=replace-with-a-long-random-token
PUBLIC_BACKEND_URL=http://localhost:8000

GITHUB_TRIGGER_TOKEN=github_pat_or_fine_grained_token
GITHUB_WORKFLOW_FILE=securegate.yml

SLACK_WEBHOOK_URL=https://hooks.slack.com/services/...
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=your-email@example.com
SMTP_PASSWORD=your-app-password
```

Copyright © 2026 SecureGate AI

```
SMTP_FROM=your-email@example.com

SECURITY_SCORE_THRESHOLD=80
CRITICAL_VULNERABILITY_THRESHOLD=0
HIGH_VULNERABILITY_THRESHOLD=5
BLOCK_HIGH_VULNERABILITIES=true
QUALITY_GATE_REQUIRED=true
SECRET_DETECTION_REQUIRED=true

PROMETHEUS_ENABLED=true
ENABLE_MONITORING=true
ENABLE_ANALYTICS=true
ALERTMANAGER_URL=http://alertmanager:9093
ALERT_ESCALATION_MINUTES=30

AI_ENABLED=true
OLLAMA_BASE_URL=http://ollama:11434
OLLAMA_MODEL=gemma3:4b
OLLAMA_TIMEOUT_SECONDS=300
```

For ngrok local testing:

```
PUBLIC_BACKEND_URL=https://your-ngrok-domain.ngrok-free.dev
```

Use the same public URL in GitHub repository secrets as `SECUREGATE_URL`.

Validation

Backend Syntax Validation

```
python3 -m compileall -q backend/app
```

Frontend Build Validation

```
cd frontend
npm run build
```

Docker Compose Validation

```
docker compose config --quiet
```

Health Check

```
curl -fsS http://localhost:8000/health
```

Metrics Check

```
curl -fsS http://localhost:8000/metrics
```

Swagger API Documentation

Open:

```
http://localhost:8000/docs
```

End-To-End Validation Checklist

Copyright © 2026 SecureGate AI

Validation Item	Expected Result
Frontend opens	Dashboard loads at `http://localhost:5173`.
Backend health	`/health` returns healthy.
PostgreSQL running	Backend can store projects, scans, findings, alerts, and reports.
Project registration	New repository creates one project ID. Duplicate repository is rejected.
GitHub Actions scan	Pipeline uploads reports and completes scan lifecycle.
Report generation	Master PDF, DOCX, JSON, tool PDFs, and AI PDFs are downloadable.
Security decision	Approved/rejected status matches configured policy.
Alerts page	Default policy and custom project policy are visible.
Notifications	Slack and email messages are sent when configured.
Summary workflow	Daily, weekly, and monthly summary endpoints can be triggered by GitHub Actions cron.
Grafana	Dashboard loads and displays SecureGate AI metrics.
AI Assistant	Ollama answers when available; fallback works when unavailable.

Operational Runbook

Register A Project

1. Open `http://localhost:5173/projects`.
2. Enter project name and repository URL.
3. Submit the project.
4. Confirm SecureGate AI generated a project ID.
5. Open the Manifests page and copy the required pipeline manifest.
6. Add the manifest to the repository.

Trigger A Scan

A scan can be triggered by:

- Pushing code to a configured branch.
- Running GitHub Actions workflow manually.
- Using the Trigger Scan button from the SecureGate AI project page when GitHub trigger credentials are configured.

Review A Failed Scan

1. Open the project page.
2. Review Security Summary.
3. Open the latest scan.

Copyright © 2026 SecureGate AI

4. Check Uploaded Reports and Normalized Findings.
5. Download the master report.
6. Open the Remediation Plan page.
7. Download the AI remediation plan.
8. Fix issues by P1-P7 priority.
9. Push updated code to trigger a new scan.

Manage Alerts

1. Open Alerts.
2. Review Alert Routing Policy for default rules.
3. Use Custom Alert Management for project-specific routing.
4. Set Slack and email severity thresholds.
5. Save policy.
6. Future alerts for that project follow the custom policy.

Review Monitoring

1. Open Monitoring page in SecureGate AI for service dependency status.
2. Open Grafana at `http://localhost:3000` for metrics visualization.
3. Open Prometheus at `http://localhost:9090` for raw metrics and target health.

References

Official documentation references:

- FastAPI: https://fastapi.tiangolo.com/
- React: https://react.dev/
- Vite: https://vitejs.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- PostgreSQL: https://www.postgresql.org/docs/
- Docker Compose: https://docs.docker.com/compose/
- GitHub Actions: https://docs.github.com/actions
- Jenkins Pipeline: https://www.jenkins.io/doc/book/pipeline/
- GitLeaks: https://github.com/gitleaks/gitleaks
- Semgrep: https://semgrep.dev/docs/
- Trivy: https://aquasecurity.github.io/trivy/
- SonarQube: https://docs.sonarsource.com/sonarqube-server/
- SonarCloud: https://docs.sonarsource.com/sonarcloud/
- Prometheus: https://prometheus.io/docs/
- Grafana: https://grafana.com/docs/grafana/latest/

Copyright © 2026 SecureGate AI

- Alertmanager: https://prometheus.io/docs/alerting/latest/alertmanager/
- Slack Incoming Webhooks: https://api.slack.com/messaging/webhooks
- Ollama: https://github.com/ollama/ollama

Summary

SecureGate AI is a complete DevSecOps security decision platform that converts CI/CD scanner outputs into clear deployment decisions, reports, alerts, dashboards, and remediation plans. It supports GitHub Actions and Jenkins, uses PostgreSQL for persistent scan history, exposes Prometheus metrics, visualizes operational data in Grafana, and adds local AI assistance through Ollama and Gemma.

The platform's main value is consolidation. It brings secrets, vulnerabilities, code quality, unit tests, Docker image risk, notifications, reports, monitoring, and remediation guidance into one controlled dashboard. This helps teams detect security issues earlier, make consistent deployment decisions, and reduce the time required to understand and fix vulnerabilities.

SecureGate AI is ready as a capstone-grade implementation through Sprint 6 and provides a strong foundation for future enterprise improvements such as RBAC, cloud deployment, deeper Grafana drilldowns, advanced AI remediation, and compliance-oriented reporting.

Author

Saima Usman \

Jr. DevOps + Cloud Engineer \

HeroVired (PPMCAD-15)

Capstone Project \

Container Image Vulnerability Scanner with Reporting

Project Name: \

SecureGate AI

Copyright © 2026 SecureGate AI